John Resig

Server-Side JavaScript with Jaxer

Imagine ripping off the visual rendering part of Firefox and replacing it with a hook to Apache instead – roughly speaking that's what Jaxer is.

Jaxer is a strange new hybrid application coming from the folks at Aptana (the makers of the quite-excellent web IDE). Literally, they took the Mozilla platform (which powers Firefox, Camino, etc.) and built a stateful platform on top of it.

You can imagine it as a browser that's able to render pages, modify them dynamically with JavaScript and the DOM, and even use JavaScript libraries with great ease – with the exception that it's living as a standalone server-side process, un-associated with the typical browsing experience.

Aptana has implemented something that's greatly interested me ever since I started playing with JavaScript on the server-side (via Rhino). My friends over at AppJet even took a step towards making this concept easy, hosted, and free. The difference here is the severe amount of power and leverage that you're gaining with this platform: file I/O, network connectivity, database access, and even email access.

Let's back up a bit and look at an example of a simple, but representative, Jaxer application. This simple application is a textarea which, when submitted, quietly saves the results to a text file on the server. What you see below is the entire application – there is no other server-side language being used save for JavaScript.

```
<html>
<head>
  <script src="http://code.jquery.com/jquery.js" runat="both"></script>
  <script>
    jQuery(function($){
      $("form").submit(function(){
        save( $("textarea").val() );
        return false;
      });
    });
  </script>
  <script runat="server">
    function save( text ){
      Jaxer.File.write("tmp.txt", text);
    }
    save.proxy = true;

    function load(){
      $("textarea").val(
        Jaxer.File.exists("tmp.txt") ? Jaxer.File.read("tmp.txt") : "");
```

```
      }
    </script>
  </head>
  <body onserverload="load()">
    <form action="" method="post">
      <textarea></textarea>
      <input type="submit"/>
    </form>
  </body>
  </html>
```

There's a bunch of key points here that we need to take a look at in order to understand what's going on here, however it's most important to note that this application looks just like a normal HTML/JavaScript/DOM web app – there are no significant differences in syntax or style. Let's dig in and see what all there is at play here.

- `runat="both"` and `runat="server"` – These script blocks are just like normal script blocks, but with an important exception: They both are capable of running on the server, rather than on the client. It's important to remember that since a Jaxer server is just like a headless Firefox, thus it's able to handle the same JavaScript that Firefox can. In this case that's both the whole jQuery library and a couple of helper functions. The `runat="both"` declaration, additionally, states that the script block should be executed on both the client and the server (in this demo we're using jQuery in both places).
- `Jaxer.File` – These set of methods are part of the Jaxer API. It's important to note that these functions can only be run on the server-side, thus they should only be called in a server-capable script block. The methods do what you would expect them to: They allow you to do simple interactions with text files.
- `save.proxy = true;` – The save() function must exist on the server-side (since it contains calls to server-side functionality) however we want to be able to call it from the client-side whenever the user hits the submit button to save their textarea changes. We can do this by making it a proxy function (adding an extra proxy property to the function). This actually creates a stub function on the client-side, to allow the code to continue working as it would expect, while obscuring the actual functionality behind an Ajax request. It's all completely seamless and works as you would expect it to, which is impressive. You can also make functions proxy functions by using a `runat="proxy"` on a script block.
- `onserverload="load()"` – This attribute gives a simple mechanism for running a piece of server-side functionality on DOM ready. In this case we're loading the contents of the saved text file and injecting it into the textarea, using jQuery.

There's also one thing that I want to mention: I built the above mini app after only a cursory glance through the examples and it worked 100% perfectly out of the box, with no errors whatsoever. This is really hard to do and even more so with the first release of a project – I'm quite impressed. Even taking nothing else into account it's pretty obvious that Jaxer is easily the most exciting piece of JavaScript technology to arrive in a long time.

Disclaimer: I'm a proud member of Aptana's advisory board. I've been greatly impressed with their work and I'm glad they asked me to join them.

Posted: January 23rd, 2008

Subscribe for email updates

29 Comments (Show Comments)

Comments are closed.
Comments are automatically turned off two weeks after the original post. If you have a question concerning the content of this post, please feel free to contact me.

Secrets of the JS Ninja

Secret techniques of top JavaScript programmers. Published by Manning.

Subscribe for email updates

@jeresig / Mastodon

Infrequent, short, updates and links.